

Comprehensive Project Report

Guwahati Smart Flood Risk Monitoring & Decision Support System

1. Executive Summary

The **Guwahati Smart Flood Risk Monitoring & Decision Support System (DSS)** is a comprehensive, full-stack predictive application designed to mitigate the severe urban flooding challenges faced by Guwahati, Assam. The city annually suffers from heavy monsoon rainfall, which, when combined with the rising water levels of the Brahmaputra River and inefficient natural drainage, results in significant waterlogging and disruptions in major areas like Anil Nagar, Zoo Road, and Beltola.

This system leverages advanced Machine Learning to predict localized flood risks in real time. It goes beyond simple predictions by offering "**Explainable AI**" mechanisms—informing the user *why* an area is at high risk—and providing actionable safety measures. The platform features a modern, reactive frontend paired with an automated FastAPI backend, positioning itself as a robust, production-ready solution for urban resilience.

2. Objectives and Scope

Objectives:

- To predict localized flood probabilities using multidimensional weather and hydrological data.
- To provide a Decision Support System that delivers clear, actionable steps for evacuation and safety.
- To display real-time insights through an interactive graphical dashboard.
- To maintain an architecture that is highly scalable, containerized, and ready for cloud deployment.
- To empower citizens through a localized flood reporting and alerting system.

Scope:

- Geographical Focus:** Key flood-prone zones in Guwahati: Beltola, Zoo Road, Anil Nagar, Bhangagarh, and Uzan Bazaar.
 - Audience:** City administrators, disaster response teams, and general residents searching for up-to-date guidance and risk factors.
-

3. Architecture & Tech Stack

3.1. Frontend Architecture

The user interface is engineered to be highly responsive, dark-themed for better visibility of data-heavy metrics, and interactive.

- Framework:** React.js powered by Vite for rapid development.
- Styling:** Tailwind CSS for a modern, utility-first design system.
- Geospatial Mapping:** Leaflet.js to plot zones directly onto interactive maps with dynamic color-coding.
- Data Visualization:** Recharts to display historical trends and future 6-hour risk forecasting.
- Real-time Communication:** WebSockets for receiving instant emergency alerts.

3.2. Backend Architecture

The backend serves as the logic and data powerhouse, designed for low-latency predictions and persistent storage.

- **Framework:** FastAPI (Python), run via Uvicorn, selected for its asynchronous capabilities and automatic validation.
 - **Machine Learning Layer:** Hybrid Ensemble Model combining **Random Forest**, **Gradient Boosting**, and **Logistic Regression** (Voting Classifier with weighted soft-voting).
 - **Database:** SQLite with SQLAlchemy ORM for managing historical weather records and citizen reports.
 - **Real-time Alerts:** WebSocket Manager to broadcast emergency notifications to all connected clients.
-

4. Machine Learning Implementation

The core intelligence of the application is a robust Ensemble Model built with Scikit-Learn.

Features Considered (Hydrological & Meteorological Parameters):

1. **River Level (m):** Current Brahmaputra level relative to the danger mark (approx. 49.68m).
2. **Rainfall Amount (mm):** Recent local precipitation measurements.
3. **Humidity (%):** Atmospheric moisture levels influencing soil saturation.
4. **Soil Moisture:** Percentage of soil saturation affecting runoff speed.
5. **Drainage Capacity:** Current effectiveness of natural and artificial drainage systems.
6. **Temperature (°C):** Local temperature impacting evaporation rates.

Risk Stratification (Target Variable):

- **Low (0):** Normal levels, no immediate threat.
- **Medium (1):** Pre-alert stage, increasing soil saturation and waterlogging.
- **High (2):** Imminent flooding risk, immediate action required.

Explainability & Actions: The system decomposes the ensemble's output probabilities to calculate confidence scores. It approximates feature contribution (e.g., "70% driven by rainfall, 30% by river levels") and dynamically generates textual justifications like, *"Critical: Heavy rainfall combined with Brahmaputra above danger mark."* It then matches this state to predefined emergency protocols to generate action items like *"Evacuate low-lying areas immediately"*.

5. Application Features Detailed

A. Interactive Map & Dashboard

A live mapping interface that color-codes Guwahati's zones based on real-time risk assessments. The dashboard provides clear metrics, including accuracy and F1 scores, and visualizes the model's performance via confusion matrices.

B. The Simulation Panel

Designed for scenario planning, this panel permits users to manually adjust rainfall, river levels, humidity, and more. The model processes the test parameters instantly, demonstrating its robustness and enabling "what-if" analysis (e.g., *"What happens if it rains 150mm with 90% soil moisture?"*).

C. Automated Forecasting & History

The system generates a 6-hour forecast using a physics-informed random-walk projection of current trends. It also maintains a 12-hour historical log to help users understand how risk levels have evolved over time.

D. Citizen Reporting & Real-time Alerts

Users can submit localized flood reports (zone, severity, description), which are stored in the database. When a high-risk scenario is detected or manually triggered, the system broadcasts an emergency alert via WebSockets to all connected users, ensuring rapid communication.

6. Installation and Deployment

The system is containerized for effortless deployment:

1. **Docker Deployment:** Use `docker-compose up --build -d` to orchestrate the entire stack (FastAPI backend, React frontend, and persistent database).
2. **Local Setup:**
 - **Backend:** Initialize a Python virtual environment, install `requirements.txt`, and run `uvicorn main:app`.
 - **Frontend:** Install Node packages in the `/frontend` directory and run `npm run dev`.

Future Scaling Goals:

- **Real-time API Integration:** Linking live data feeds from IMD or CWC.
 - **Cloud Migration:** Moving to PostgreSQL on AWS/Supabase for enterprise-grade data persistence.
 - **Mobile Application:** Developing a React Native app for push notification-based citizen alerts.
-

7. Conclusion

The Guwahati Smart Flood Risk Monitoring System showcases a remarkable intersection of modern web development and applied predictive modeling. By transforming raw meteorological abstractions into an explainable, visual framework with concrete action points, it acts as a critical tool for disaster mitigation and urban resilience in Guwahati.